Министерство науки и высшего образования Российской Федерации

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»

Институт информатики и кибернетики

Кафедра технической кибернетики

Отчет по лабораторной работе №3

Дисциплина: «ООП»

Тема «Исключения и интерфейсы»

Выполнила: Степанова А. Д.

Группа: 6201-120303D

Самара, 2025

Задание на лабораторную работу

Дополнить пакет для работы с функциями одной переменной, заданными в табличной форме, добавив классы исключений, новый класс функций и базовый интерфейс.

Задание 1

Ознакомилась со следующими классами исключений, входящих в API Java:

- `java.lang.Exception`;
- `java.lang.IndexOutOfBoundsException`;
- `java.lang.ArrayIndexOutOfBoundsException`;
- `java.lang.IllegalArgumentException`;
- `java.lang.IllegalStateException`.

Задание 2

В пакете functions создала два класса исключений:

• FunctionPointIndexOutOfBoundsException – исключение выхода за границы набора точек при обращении к ним по номеру, наследует от класса IndexOutOfBoundsException;

```
1   package functions;
2
3   public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException {
4       public FunctionPointIndexOutOfBoundsException() {  no usages
5           super();
6       }
7
8       public FunctionPointIndexOutOfBoundsException(String message) {  13 usages
9           super(message);
10      }
11  }
```

• InappropriateFunctionPointException – исключение, выбрасываемое при попытке добавления или изменения точки функции несоответствующим образом, наследует от класса Exception.

```java
package functions;

public class InappropriateFunctionPointException extends Exception {
    public InappropriateFunctionPointException() {  no usages
        super();
    }

    public InappropriateFunctionPointException(String message) {  10 us
        super(message);
    }
}

```

Задание 3

В разработанный ранее класс TabulatedFunction внесла изменения, обеспечивающие выбрасывание исключений методами класса.

Оба конструктора класса должны выбрасывать исключение IllegalArgumentException, если левая граница области определения больше или равна правой, а также если предлагаемое количество точек меньше двух.

```java
public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount) {  2 usages

    if (doubleGreaterOrEq(leftX, rightX)) {
        throw new IllegalArgumentException("Левая граница не может быть больше или равна правой");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException("Количество точек не может быть меньше 2");
    }

    this.pointsCount = pointsCount;
    this.points = new FunctionPoint[pointsCount + 10];

    double step = (rightX - leftX) / (pointsCount - 1);

    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        points[i] = new FunctionPoint(x,  y: 0);
    }
}
```

```java
public ArrayTabulatedFunction(double leftX, double rightX, double[] values) {  no usages

    if (doubleGreaterOrEq(leftX, rightX)) {
        throw new IllegalArgumentException("Левая граница не может быть больше или равна правой");
    }
    if (values.length < 2) {
        throw new IllegalArgumentException("Количество точек не может быть меньше 2");
    }

    this.pointsCount = values.length;
    this.points = new FunctionPoint[values.length + 10];

    double step = (rightX - leftX) / (values.length - 1);

    for (int i = 0; i < values.length; i++) {
        double x = leftX + i * step;
        points[i] = new FunctionPoint(x, values[i]);
    }
}
```

Методы getPoint(), getPointX(), getPointY(), setPointY() выбрасывают исключение FunctionPointIndexOutOfBoundsException, если переданный в метод номер выходит за границы набора точек.

```java
//Метод возвращает копию точки, соответствующей данному индексу
public FunctionPoint getPoint(int index) {  5 usages
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " выходит за границы. ");
    }
    return new FunctionPoint(points[index]);
}
```

```java
//Метод возвращает значение абсциссы точки с указанным номером
public double getPointX(int index) {  24 usages
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " выходит за границы. ");
    }
    return points[index].getX();
}
```

```java
//Метод возвращает значение ординаты точки с указанным номером
public double getPointY(int index) {  14 usages
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " выходит за границы.");
    }
    return points[index].getY();
}
```

```java
//Метод изменяет значение ординаты точки с указанным номером
public void setPointY(int index, double y) {  3 usages
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " выходит за границы. ");
    }
    points[index] = new FunctionPoint(getPointX(index), y);
}
```

Методы setPoint(), setPointX() выбрасывают исключение FunctionPointIndexOutOfBoundsException, если переданный в метод номер

выходит за границы набора точек, и исключение InappropriateFunctionPointException в том случае, если координата x задаваемой точки лежит вне интервала, определяемого значениями соседних точек табулированной функции.

```
94      //Метод заменяет указанную точку на переданную
95      public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {  3 usages
96
97          if (index < 0 || index >= pointsCount) {
98              throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " выходит за границы. ");
99          }
100
101         double newX = point.getX();
102
103         //если newX меньше левой границы
104         if (index != 0 && doubleLessOrEq(newX,getPointX( index: index - 1))) {
105             throw new InappropriateFunctionPointException("X новой точки должен быть больше X предыдущей точки");
106         }
107
108         //если newX больше правой границы
109         if (index != pointsCount - 1 && doubleGreaterOrEq(newX,getPointX( index: index + 1))) {
110             throw new InappropriateFunctionPointException("X новой точки должен быть меньше X следующей точки");
111         }
112         //если X лежит в нужном интервале, создаем новую точку
113         points[index] = new FunctionPoint(point);
114     }
```

```
129     //Метод изменяет значение абсциссы точки с указанным номером
130     public void setPointX(int index, double x) throws InappropriateFunctionPointException {  2 usages
131         if (index < 0 || index >= pointsCount) {
132             throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " выходит за границы. ");
133         }
134         //если x меньше левой границы
135         if (index != 0 && doubleLessOrEq(x,getPointX( index: index - 1))) {
136             throw new InappropriateFunctionPointException("X новой точки должен быть больше X предыдущей точки");
137         }
138         //если x больше правой границы
139         if (index != pointsCount - 1 && doubleGreaterOrEq(x,getPointX( index: index + 1))) {
140             throw new InappropriateFunctionPointException("X новой точки должен быть меньше X следующей точки");
141         }
142         //создаем новую точку с новым X и старым Y
143         points[index] = new FunctionPoint(x, getPointY(index));
144     }
```

Метод addPoint() выбрасывает исключение InappropriateFunctionPointException, если в наборе точек функции есть точка, абсцисса которой совпадает с абсциссой добавляемой точки.

```java
//Метод добавления новой точки
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {  4 usages

    for (int i = 0; i < pointsCount; i++) {
        if (doubleEq(point.getX(),getPointX(i))) {
            throw new InappropriateFunctionPointException("Точка с таким X уже существует");
        }
    }
    int searchIndex = 0;
    while (searchIndex < pointsCount && doubleGreater(point.getX(),getPointX(searchIndex))) {
        searchIndex++;
    }
    if (pointsCount == points.length) {
        FunctionPoint[] newPoints = new FunctionPoint[points.length * 2];
        System.arraycopy(points,  srcPos: 0, newPoints,  destPos: 0, pointsCount);
        points = newPoints;
    }
    System.arraycopy(points, searchIndex, points,  destPos: searchIndex + 1,  length: pointsCount - searchIndex);
    points[searchIndex] = new FunctionPoint(point);
    pointsCount++;
}
```

Метод deletePoint() выбрасывает исключение IllegalStateException, если на момент удаления точки количество точек в наборе менее трех.

```java
    //Метод удаления точки функции
    public void deletePoint(int index) {  2 usages
        if (pointsCount < 3) {
            throw new IllegalStateException("Невозможно удалить точку, т.к. в наборе должно быть не менее двух точек");
        }
        if (index < 0 || index >= pointsCount) {
            throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " выходит за границы. ");
        }
        System.arraycopy(points,  srcPos: index + 1, points, index,  length: pointsCount - index - 1);
        pointsCount--;
        points[pointsCount] = null;
    }
```

Задание 4

В пакете functions создала класс LinkedListTabulatedFunction.

Внутри класса LinkedListTabulatedFunction создплп класс элементов списка FunctionNode, содержащий информационное поле для хранения данных типа FunctionPoint, а также поля для хранения ссылок на предыдущий и следующий элемент. FunctionNode описан как private внутренний класс, поскольку FunctionNode является вспомогательной структурой, используемой для внутренней организации связного списка в LinkedListTabulatedFunction, поэтому он скрыт от внешнего кода. Это предотвращает прямое создание и манипулирование узлами извне, что защищает целостность структуры данных.

```
public class LinkedListTabulatedFunction implements TabulatedFunction {  5 usages
    private class FunctionNode {  22 usages
        private FunctionPoint point;  18 usages
        private FunctionNode prev;  16 usages
        private FunctionNode next;  20 usages

        public FunctionNode() {  3 usages
            this.point = null;
            this.prev = null;
            this.next = null;
        }

        public FunctionNode(FunctionPoint point, FunctionNode prev, FunctionNode next) {
            this.point = point;
            this.prev = prev;
            this.next = next;
        }
    }

```

Поля класса LinkedListTabulatedFunction: поле ссылки на объект головы, поле количества элементов в списке, private FunctionNode lastNode – запомненный узел, к которому было последнее обращение, private int lastIndex – его индекс.

Для класса LinkedListTabulatedFunction описала конструктор по умолчанию, который инициализирует структуру двусвязного кольцевого списка.

```
    private FunctionNode head;  14 usages
    private int pointsCount;  20 usages

    //последний узел к которому обращались
    private FunctionNode lastNode;  3 usages
    private int lastIndex;  8 usages

    public LinkedListTabulatedFunction() {  2 usages
        head = new FunctionNode();

        head.next = head;
        head.prev = head;

        pointsCount = 0;
        lastNode = head;
        lastIndex = -1;
    }
```

В классе LinkedListTabulatedFunction реализовала следующие методы:

- FunctionNode getNodeByIndex(int index), возвращающий ссылку на объект элемента списка по его номеру.

В данном методе реализована оптимизация доступа к элементам двусвязного циклического списка:

если запрашивается элемент, соседний к последнему полученному, то поиск начинается с lastNode и идёт по соседним узлам;

если индекс находится в первой половине списка, обход начинается с головы (head.next) и движется вперёд;

если индекс во второй половине — обход начинается с конца (head.prev) и движется назад.

```
40      //возвращает ссылку на узел списка по его номеру
41      private FunctionNode getNodeByIndex(int index) {  8 usages
42
43          FunctionNode currentNode;
44          //с запомненного элемента
45          if (lastIndex != -1 && Math.abs(index - lastIndex) <= 1) {
46              currentNode = lastNode;
47              if (index > lastIndex) {
48                  for (int i = lastIndex; i < index; i++) {
49                      currentNode = currentNode.next;
50                  }
51              } else if (index < lastIndex) {
52                  for (int i = lastIndex; i > index; i--) {
53                      currentNode = currentNode.prev;
54                  }
55              }
56          }
57          //с начала списка
58          else if (index <= pointsCount / 2) {
59              currentNode = head.next;
60              for (int i = 0; i < index; i++) {
61                  currentNode = currentNode.next;
62              }
63          }
```

```
64          //с конца
65          else {
66              currentNode = head.prev;
67              for (int i = pointsCount - 1; i > index; i--) {
68                  currentNode = currentNode.prev;
69              }
70          }
71          lastNode = currentNode;
72          lastIndex = index;
73
74          return currentNode;
75      }
```

- FunctionNode addNodeToTail(), добавляющий новый элемент в конец списка и возвращающий ссылку на объект этого элемента.

```
77 @    private FunctionNode addNodeToTail() {  3 usages
78          FunctionNode newNode = new FunctionNode();
79
80          FunctionNode currentLast = head.prev;
81
82          newNode.prev = currentLast;
83          newNode.next = head;
84          currentLast.next = newNode;
85          head.prev = newNode;
86
87          pointsCount ++;
88          return newNode;
89      }
```

- FunctionNode addNodeByIndex(int index), добавляющий новый элемент в указанную позицию списка и возвращающий ссылку на объект этого элемента.

```java
//добавляет новый элемент в указанную позицию списка
private FunctionNode addNodeByIndex(int index) {  1 usage
    //в конец списка
    if (index == pointsCount) {
        return addNodeToTail();
    }

    FunctionNode newNode = new FunctionNode();
    FunctionNode currentNode = getNodeByIndex(index);

    newNode.prev = currentNode.prev;
    newNode.next = currentNode;

    currentNode.prev.next = newNode;
    currentNode.prev = newNode;

    pointsCount ++;
    return newNode;
}
```

- FunctionNode deleteNodeByIndex(int index), удаляющий элемент списка по номеру и возвращающий ссылку на объект удаленного элемента.

```java
//удаляет элемент списка по номеру
private FunctionNode deleteNodeByIndex(int index) {  1 usage

    FunctionNode deleteNode = getNodeByIndex(index);

    deleteNode.prev.next = deleteNode.next;
    deleteNode.next.prev = deleteNode.prev;
    pointsCount --;
    return deleteNode;
}
```

Задание 5

Реализовать в классе LinkedListTabulatedFunction конструкторы и методы, аналогичные конструкторам и методам класса TabulatedFunction.

```java
    public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount) {  3 usages
        this();

        if (doubleGreaterOrEq(leftX, rightX)) {
            throw new IllegalArgumentException("Левая граница не может быть больше или равна правой");
        }
        if (pointsCount < 2) {
            throw new IllegalArgumentException("Количество точек не может быть меньше 2");
        }

        double step = (rightX - leftX) / (pointsCount - 1);
        for (int i = 0; i < pointsCount; i++) {
            double x = leftX + i * step;
            addNodeToTail().point = new FunctionPoint(x,  y: 0);
        }
    }
    public LinkedListTabulatedFunction(double leftX, double rightX, double[] values) {  1 usage
        this();

        if (doubleGreaterOrEq(leftX, rightX)) {
            throw new IllegalArgumentException("Левая граница не может быть больше или равна правой");
        }
        if (values.length < 2) {
            throw new IllegalArgumentException("Количество точек не может быть меньше 2");
        }

        double step = (rightX - leftX) / (values.length - 1);
        for (int i = 0; i < values.length; i++) {
            double x = leftX + i * step;
            addNodeToTail().point = new FunctionPoint(x, values[i]);
        }
    }
```

```
152     public double getLeftDomainBorder() {  3 usages
153         return  getPointX( index: 0);
154     }
155     public double getRightDomainBorder() {  3 usages
156         return getPointX( index: pointsCount - 1);
157     }
158     //Возвращает значение функции в точке x, если точка лежит в обл. определения
159     public double getFunctionValue(double x) {  2 usages
160         if (doubleLess(x,getLeftDomainBorder()) || doubleGreater(x,getRightDomainBorder())) {
161             return Double.NaN;
162         }
163         FunctionNode current = head.next;
164         for (int i = 0; i < pointsCount - 1; i++) {
165             double x1 = current.point.getX();
166             double x2 = current.next.point.getX();
167 
168             if (doubleGreaterOrEq(x, x1) && doubleLessOrEq(x, x2)) {
169                 if (doubleEq(x, x1)) {
170                     return  current.point.getY();
171                 }
172                 else if (doubleEq(x, x2)) {
173                     return  current.next.point.getY();
174                 }
175                 else {
176                     return interpolation(x, current.point, current.next.point);
177                 }
178             }
179             current = current.next;
180         }
181         return Double.NaN;
182     }
183     // Линейная интерполяция
184 @   private double interpolation(double x, FunctionPoint p1, FunctionPoint p2) {  1 usage
185         return p1.getY() + (x - p1.getX()) * (p2.getY() - p1.getY()) / (p2.getX() - p1.getX());
186     }
```

```
188     public int getPointsCount() {  3 usages
189         return pointsCount;
190     }
191     //Метод возвращает копию точки, соответствующей данному индексу
192     public FunctionPoint getPoint(int index) {  5 usages
193         if (index < 0 || index >= pointsCount) {
194             throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " выходит за границы.");
195         }
196         return new FunctionPoint(getNodeByIndex(index).point);
197     }
```

```java
//Метод заменяет указанную точку на переданную
public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {  3 usages

    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " выходит за границы.");
    }
    double newX = point.getX();
    //если newX меньше левой границы
    if (index != 0 && doubleLessOrEq(newX,getPoint( index: index - 1).getX())) {
        throw new InappropriateFunctionPointException("X новой точки должен быть больше X предыдущей точки");
    }
    //если newX больше правой границы
    if (index != pointsCount - 1 && doubleGreaterOrEq(newX,getPoint( index: index + 1).getX())) {
        throw new InappropriateFunctionPointException("X новой точки должен быть меньше X следующей точки");
    }
    getNodeByIndex(index).point = new FunctionPoint(point);
}
```

```java
//Метод возвращает значение абсциссы точки с указанным номером
public double getPointX(int index) {  24 usages
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " выходит за границы.");
    }
    return getNodeByIndex(index).point.getX();
}
//Метод возвращает значение ординаты точки с указанным номером
public double getPointY(int index) {  14 usages
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " выходит за границы.");
    }
    return getNodeByIndex(index).point.getY();
}
```

```java
//Метод изменяет значение абсциссы точки с указанным номером
public void setPointX(int index, double x) throws InappropriateFunctionPointException {  2 usages
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " выходит за границы.");
    }
    if (index != 0 && doubleLessOrEq(x,getPoint( index: index - 1).getX())) {
        throw new InappropriateFunctionPointException("X новой точки должен быть больше X предыдущей точки");
    }
    if (index != pointsCount - 1 && doubleGreaterOrEq(x,getPoint( index: index + 1).getX())) {
        throw new InappropriateFunctionPointException("X новой точки должен быть меньше X следующей точки");
    }
    getNodeByIndex(index).point = new FunctionPoint(x, getPointY(index));
}
//Метод изменяет значение ординаты точки с указанным номером
public void setPointY(int index, double y) {  3 usages
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " выходит за границы.");
    }
    getNodeByIndex(index).point = new FunctionPoint(getPointX(index), y);
}
```

```java
    //Метод удаления точки функции
    public void deletePoint(int index) {  2 usages
        if (pointsCount < 3) {
            throw new IllegalStateException("Невозможно удалить точку, т.к. в наборе должно быть не менее двух точек" );
        }
        deleteNodeByIndex(index);
    }
    //Метод добавления новой точки
    public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {  4 usages

        for (int i = 0; i < pointsCount; i++) {
            if (doubleEq(point.getX(),getPointX(i))) {
                throw new InappropriateFunctionPointException("Точка с таким X уже существует");
            }
        }
        int searchIndex = 0;
        FunctionNode current = head.next;
        while (current != head && doubleGreater(point.getX(), current.point.getX())) {
            current = current.next;
            searchIndex++;
        }
        FunctionNode newNode = addNodeByIndex(searchIndex);
        newNode.point = new FunctionPoint(point);
    }
}
```

Задание 6

Класс TabulatedFunction переименовала в класс ArrayTabulatedFunction.

Создала интерфейс TabulatedFunction, содержащий объявления общих методов классов ArrayTabulatedFunction и LinkedListTabulatedFunction.

```java
package functions;

public interface TabulatedFunction {  9 usages  2 implementations

    int getPointsCount();  3 usages  2 implementations
    double getLeftDomainBorder();  3 usages  2 implementations
    double getRightDomainBorder();  3 usages  2 implementations
    double getFunctionValue(double x);  2 usages  2 implementations
    FunctionPoint getPoint(int index);  5 usages  2 implementations
    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException;
    double getPointX(int index);  24 usages  2 implementations
    void setPointX(int index, double x) throws InappropriateFunctionPointException;  2 usages  2 im
    double getPointY(int index);  14 usages  2 implementations
    void setPointY(int index, double y);  3 usages  2 implementations
    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;  4 usages  2 imp
    void deletePoint(int index);  2 usages  2 implementations
}
```

Оба класса функций реализовывают созданный интерфейс:

public class ArrayTabulatedFunction implements TabulatedFunction {…}

public class LinkedListTabulatedFunction implements TabulatedFunction {…}

Задание 7

Класс Main

```java
import functions.FunctionPointIndexOutOfBoundsException;
import functions.InappropriateFunctionPointException;
import functions.FunctionPoint;
import functions.ArrayTabulatedFunction;
import functions.LinkedListTabulatedFunction;
import functions.TabulatedFunction;

public class Main {
    public static void main(String[] args) {

        System.out.println("--- Тестирование через интерфейс TabulatedFunction ---");

        // Тестирование ArrayTabulatedFunction
        System.out.println("\n--- ArrayTabulatedFunction ---");
        testTabulatedFunction(new ArrayTabulatedFunction( leftX: 0,  rightX: 6,  pointsCount: 10));

        // Тестирование LinkedListTabulatedFunction
        System.out.println("\n--- LinkedListTabulatedFunction ---");
        testTabulatedFunction(new LinkedListTabulatedFunction( leftX: 0,  rightX: 6,  pointsCount: 10));

        // Дополнительное тестирование исключений
        System.out.println("\n--- Тестирование исключений ---");
        testExceptions();
    }

    public static void testTabulatedFunction(TabulatedFunction f) {  2 usages

        System.out.println("Функция f(x) = 2x + 1");
        System.out.println("Область определения: [" + f.getLeftDomainBorder() + "; "
                + f.getRightDomainBorder() + "]");
        System.out.println("Количество точек: " + f.getPointsCount());
        System.out.println("\nТочки функции: ");

        for (int i = 0; i < f.getPointsCount(); i++) {
            f.setPointY(i,  y: 2 * f.getPointX(i) + 1);
            System.out.printf("%d: (%.2f; %.2f)%n", i+1, f.getPointX(i), f.getPointY(i));
        }
```

```java
        // Проверка метода getFunctionValue
        System.out.println("\nЗначения функции f(x) в разных x: ");
        double[] x_val = {-1.3, 18, 1, 6, 43, 3.1};
        for (int i = 0; i < x_val.length; i++) {
            if (Double.isNaN(f.getFunctionValue(x_val[i])))
                System.out.println("Значение x = " + x_val[i] + " лежит вне области определения функции" );
            else
                System.out.printf("f(%.2f) = %.2f%n", x_val[i], f.getFunctionValue(x_val[i]));
        }

        // Проверка остальных методов
        testFunctionOperations(f);
    }
```

```
53      public static void testFunctionOperations(TabulatedFunction f) {  1 usage
54          System.out.println("\n--- Тестирование операций с точками ---");
55
56          try {
57              // Проверка setPoint #1
58              System.out.println("\nЗамена точки №3 на точку (2.5; 6)");
59              FunctionPoint test1 = new FunctionPoint( x: 2.5,  y: 6);
60              f.setPoint( index: 2, test1);
61              System.out.printf("Точка %d: (%.2f; %.2f)%n", 3, f.getPointX( index: 2), f.getPointY( index: 2));
62          } catch (Exception e) {
63              System.out.println(e);
64          }
65          try {
66              // Проверка setPoint #2
67              System.out.println("\nЗамена точки №8 на точку (5; 11)");
68              FunctionPoint test2 = new FunctionPoint( x: 5,  y: 11);
69              f.setPoint( index: 7, test2);
70              System.out.printf("Точка %d: (%.2f; %.2f)%n", 8, f.getPointX( index: 7), f.getPointY( index: 7));
71          } catch (Exception e) {
72              System.out.println(e);
73          }
74          try {
75              // Проверка setPoint #3
76              System.out.println("\nЗамена точки №25 на точку (12; 7)");
77              FunctionPoint test3 = new FunctionPoint( x: 12,  y: 7);
78              f.setPoint( index: 24, test3);
79              System.out.printf("Точка %d: (%.2f; %.2f)%n", 25, f.getPointX( index: 24), f.getPointY( index: 24));
80          } catch (Exception e) {
81              System.out.println(e);
82          }
83          try {
84              // Проверка getPointX #1
85              System.out.println("\nАбсцисса точки №16");
86              System.out.println(f.getPointX( index: 15));
87          } catch (Exception e) {
88              System.out.println(e);
89          }
```

```
90          try {
91              // Проверка getPointX #2
92              System.out.println("\nАбсцисса точки №1");
93              System.out.println(f.getPointX( index: 0));
94          } catch (Exception e) {
95              System.out.println(e);
96          }
97          try {
98              // Проверка getPointY #1
99              System.out.println("\nОрдината точки №16");
100             System.out.println(f.getPointY( index: 15));
101         } catch (Exception e) {
102             System.out.println(e);
103         }
104         try {
105             // Проверка getPointY #3
106             System.out.println("\nОрдината точки №1");
107             System.out.println(f.getPointY( index: 0));
108         } catch (Exception e) {
109             System.out.println(e);
110         }
111         try {
```

```
        try {
            // Проверка setPointX #1
            System.out.println("\nЗамена X точки №4 на 1.1");
            f.setPointX( index: 3,  x: 1.1);
            System.out.printf("Точка %d: (%.2f; %.2f)%n", 4, f.getPointX( index: 3), f.getPointY( index: 3));
        } catch (Exception e) {
            System.out.println(e);
        }
        try {
            // Проверка setPointX #2
            System.out.println("\nЗамена X точки №9 на 5.5");
            f.setPointX( index: 8,  x: 5.5);
            System.out.printf("Точка %d: (%.2f; %.2f)%n", 9, f.getPointX( index: 8), f.getPointY( index: 8));
        } catch (Exception e) {
            System.out.println(e);
        }
        try {
            // Проверка setPointY #1
            System.out.println("\nЗамена Y точки №9 на 12");
            f.setPointY( index: 8, y: 12);
            System.out.printf("Точка %d: (%.2f; %.2f)%n", 9, f.getPointX( index: 8), f.getPointY( index: 8));
        } catch (Exception e) {
            System.out.println(e);
        }
        try {
            // Проверка setPointY #2
            System.out.println("\nЗамена Y точки №21 на 8.5");
            f.setPointY( index: 20,  y: 8.5);
            System.out.printf("Точка %d: (%.2f; %.2f)%n", 21, f.getPointX( index: 20), f.getPointY( index: 20));
        } catch (Exception e) {
            System.out.println(e);
        }
```

```
        try {
            // Проверка deletePoint
            System.out.printf("\nУдаление точки №%d: (%.2f; %.2f)%n", 5, f.getPointX( index: 4), f.getPointY( index: 4));
            f.deletePoint( index: 4);
            System.out.println("Удаление прошло успешно");

        } catch (Exception e) {
            System.out.println(e);
        }
        try {
            // Проверка addPoint #1
            System.out.println("\nДобавление точки (4, 9)");
            FunctionPoint newPoint = new FunctionPoint( x: 4,  y: 9);
            f.addPoint(newPoint);
            System.out.println("Добавление прошло успешно");

        } catch (Exception e) {
            System.out.println(e);
        }
```

```
        try {
            // Проверка addPoint #2
            System.out.println("\nДобавление точки (3, 7)");
            FunctionPoint newPoint = new FunctionPoint( x: 3,  y: 7);
            f.addPoint(newPoint);
            System.out.println("Добавление прошло успешно");
        } catch (Exception e) {
            System.out.println(e);
        }
        try {
            // Проверка addPoint #3
            System.out.println("\nДобавление точки (10, 21)");
            FunctionPoint newPoint = new FunctionPoint( x: 10,  y: 21);
            f.addPoint(newPoint);
            System.out.println("Добавление прошло успешно");
        } catch (Exception e) {
            System.out.println(e);
        }

        // Вывод итоговых точек
        System.out.println("\nИтоговые точки функции: ");
        for (int i = 0; i < f.getPointsCount(); i++) {
            System.out.printf("%d: (%.2f; %.2f)%n", i+1, f.getPointX(i), f.getPointY(i));
        }
    }
```

```
    public static void testExceptions() {  1 usage
        System.out.println("\n--- Тестирование исключений ---");

        //Некорректный конструктор
        try {
            TabulatedFunction f = new ArrayTabulatedFunction( leftX: 10,  rightX: 0,  pointsCount: 5);
        } catch (IllegalArgumentException e) {
            System.out.println(e);
        }

        //Выход индекса за границы
        try {
            TabulatedFunction f = new LinkedListTabulatedFunction( leftX: 0,  rightX: 10,  pointsCount: 3);
            f.getPoint( index: 10);
        } catch (FunctionPointIndexOutOfBoundsException e) {
            System.out.println(e);
        }

        //Удаление когда мало точек
        try {
            TabulatedFunction f = new LinkedListTabulatedFunction( leftX: 0,  rightX: 10,  pointsCount: 2);
            f.deletePoint( index: 0);
        } catch (IllegalStateException e) {
            System.out.println(e);
        }
```

```
214             //Добавление уже существующей
215             try {
216                 double[] y_values = {17, 5, -7, -19};
217                 TabulatedFunction f = new LinkedListTabulatedFunction( leftX: -2,  rightX: 4, y_values);
218                 FunctionPoint newPoint = new FunctionPoint( x: 0,  y: 9);
219                 f.addPoint(newPoint);
220             } catch (InappropriateFunctionPointException e) {
221                 System.out.println(e);
222             }
223         }
224     }
```

Выводы

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Co
--- Тестирование через интерфейс TabulatedFunction ---

--- ArrayTabulatedFunction ---
Функция f(x) = 2x + 1
Область определения: [0.0; 6.0]
Количество точек: 10

Точки функции:
1: (0,00; 1,00)
2: (0,67; 2,33)
3: (1,33; 3,67)
4: (2,00; 5,00)
5: (2,67; 6,33)
6: (3,33; 7,67)
7: (4,00; 9,00)
8: (4,67; 10,33)
9: (5,33; 11,67)
10: (6,00; 13,00)

Значения функции f(x) в разных x:
Значение x = -1.3 лежит вне области определения функции
Значение x = 18.0 лежит вне области определения функции
f(1,00) = 3,00
f(6,00) = 13,00
Значение x = 43.0 лежит вне области определения функции
f(3,10) = 7,20

--- Тестирование операций с точками ---

Замена точки №3 на точку (2.5; 6)
functions.InappropriateFunctionPointException: X новой точки должен быть меньше X следующей точки

Замена точки №8 на точку (5; 11)
Точка 8: (5,00; 11,00)

Замена точки №25 на точку (12; 7)
functions.FunctionPointIndexOutOfBoundsException: Индекс 24 выходит за границы.
```

```
Абсцисса точки №16
functions.FunctionPointIndexOutOfBoundsException: Индекс 15 выходит за границы.

Абсцисса точки №1
0.0

Ордината точки №16
functions.FunctionPointIndexOutOfBoundsException: Индекс 15 выходит за границы.

Ордината точки №1
1.0

Замена X точки №4 на 1.1
functions.InappropriateFunctionPointException: X новой точки должен быть больше X предыдущей точки

Замена X точки №9 на 5.5
Точка 9: (5,50; 11,67)

Замена Y точки №9 на 12
Точка 9: (5,50; 12,00)

Замена Y точки №21 на 8.5
functions.FunctionPointIndexOutOfBoundsException: Индекс 20 выходит за границы.

Удаление точки №5: (2,67; 6,33)
Удаление прошло успешно

Добавление точки (4, 9)
functions.InappropriateFunctionPointException: Точка с таким X уже существует

Добавление точки (3, 7)
Добавление прошло успешно

Добавление точки (10, 21)
Добавление прошло успешно
```

```
Итоговые точки функции:
1: (0,00; 1,00)
2: (0,67; 2,33)
3: (1,33; 3,67)
4: (2,00; 5,00)
5: (3,00; 7,00)
6: (3,33; 7,67)
7: (4,00; 9,00)
8: (5,00; 11,00)
9: (5,50; 12,00)
10: (6,00; 13,00)
11: (10,00; 21,00)

--- LinkedListTabulatedFunction ---
Функция f(x) = 2x + 1
Область определения: [0.0; 6.0]
Количество точек: 10

Точки функции:
1: (0,00; 1,00)
2: (0,67; 2,33)
3: (1,33; 3,67)
4: (2,00; 5,00)
5: (2,67; 6,33)
6: (3,33; 7,67)
7: (4,00; 9,00)
8: (4,67; 10,33)
9: (5,33; 11,67)
10: (6,00; 13,00)

Значения функции f(x) в разных x:
Значение x = -1.3 лежит вне области определения функции
Значение x = 18.0 лежит вне области определения функции
f(1,00) = 3,00
f(6,00) = 13,00
Значение x = 43.0 лежит вне области определения функции
f(3,10) = 7,20
```

```
--- Тестирование операций с точками ---

Замена точки №3 на точку (2.5; 6)
functions.InappropriateFunctionPointException: X новой точки должен быть меньше X следующей точки

Замена точки №8 на точку (5; 11)
Точка 8: (5,00; 11,00)

Замена точки №25 на точку (12; 7)
functions.FunctionPointIndexOutOfBoundsException: Индекс 24 выходит за границы.

Абсцисса точки №16
functions.FunctionPointIndexOutOfBoundsException: Индекс 15 выходит за границы.

Абсцисса точки №1
0.0

Ордината точки №16
functions.FunctionPointIndexOutOfBoundsException: Индекс 15 выходит за границы.

Ордината точки №1
1.0

Замена X точки №4 на 1.1
functions.InappropriateFunctionPointException: X новой точки должен быть больше X предыдущей точки

Замена X точки №9 на 5.5
Точка 9: (5,50; 11,67)

Замена Y точки №9 на 12
Точка 9: (5,50; 12,00)

Замена Y точки №21 на 8.5
functions.FunctionPointIndexOutOfBoundsException: Индекс 20 выходит за границы.
```

```
Удаление точки №5: (2,67; 6,33)
Удаление прошло успешно

Добавление точки (4, 9)
functions.InappropriateFunctionPointException: Точка с таким X уже существует

Добавление точки (3, 7)
Добавление прошло успешно

Добавление точки (10, 21)
Добавление прошло успешно

Итоговые точки функции:
1: (0,00; 1,00)
2: (0,67; 2,33)
3: (1,33; 3,67)
4: (2,00; 5,00)
5: (3,00; 7,00)
6: (3,33; 7,67)
7: (4,00; 9,00)
8: (5,00; 11,00)
9: (5,50; 12,00)
10: (6,00; 13,00)
11: (10,00; 21,00)

--- Тестирование исключений ---

--- Тестирование исключений ---
java.lang.IllegalArgumentException: Левая граница не может быть больше или равна правой
functions.FunctionPointIndexOutOfBoundsException: Индекс 10 выходит за границы.
java.lang.IllegalStateException: Невозможно удалить точку, т.к. в наборе должно быть не менее двух точе
functions.InappropriateFunctionPointException: Точка с таким X уже существует

Process finished with exit code 0
```